

In-Memory Boolean Logic Computation in Binary Neural Network Inspired Memristor Crossbar Array

Md. Abu Sayed Chowdhury

Department of Electrical and Electronic Engineering
Bangladesh University of Engineering and Technology
Dhaka-1205, Bangladesh
abu.sayed.chy76@gmail.com

Orchi Hassan

Department of Electrical and Electronic Engineering
Bangladesh University of Engineering and Technology
Dhaka-1205, Bangladesh
orchi@eee.buet.ac.bd

Abstract—Neural network hardware is often implemented using memristor crossbars for their non-volatility, high density, and parallel computation capability. In this work, we present a methodology for implementing single-bit logic computation within such memory array structures. We take a binary neural network approach to design the gates. Each binary weight is encoded using two memristor devices, and standard CMOS drivers are used for programming and readout. We demonstrate successful operation of two basic logic gates, NAND and XOR, and cascade them to demonstrate full-adder operation through circuit-level SPICE-based simulations. Any Boolean function can be implemented following this methodology to facilitate in-memory computing.

Index Terms—In-memory computing, memristor, binary neural networks

I. INTRODUCTION

The performance bottleneck posed by the von Neumann architecture has drawn significant interest in in-memory computing (IMC) paradigms, where data processing occurs directly within memory arrays. Non-conventional computing hardware architectures utilize crossbar arrays for synaptic weight storage and calculations. Segments of these arrays can be configured for in-memory computation, thereby reducing data transfer delay and mitigating the von Neumann bottleneck. While many implementations use SRAM crossbars, memristor crossbars are becoming increasingly popular due to their high density and non-volatile nature [1]–[3]. However, the full potential of memristor crossbars for performing in-memory computation within these neural network hardware implementations has seen limited exploration. In this work, we demonstrate in-memory Boolean logic computation within standard memristor crossbar arrays used in binary neural networks [4].

The binary neural network (BNN) uses binary weights and biases $\{-1, 1\}$. These weights and biases are encoded in a memristor crossbar with differential 1R synaptic cells, which employ two memristors per synapse of the BNN [5]. This configuration enables the implementation of fundamental logic gates such as NAND and XOR, which can be cascaded to form more complex circuits, such as a full-adder. The design employs CMOS drivers to program the memristors and op-amp comparators for logic sensing. The approach is validated through circuit-level SPICE simulations using the VTEAM memristor model (Fig. 2) and a 45 nm CMOS technology node [6]. This BNN-based approach demonstrates a method to use memristor crossbars to perform unified

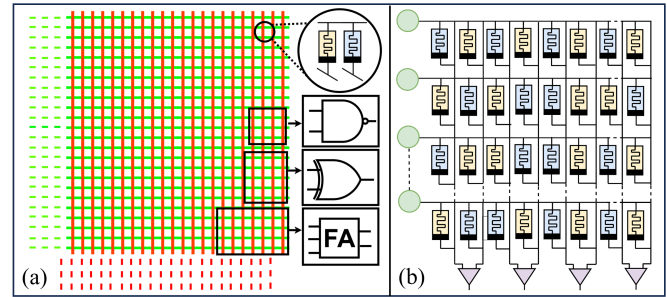


Fig. 1. In-memory computing architecture using memristor crossbar arrays: (a) Logic gates such as NAND, XOR, and full-adder can be implemented within a small portion of the crossbar array using the binary neural network approach. (b) Each cell in the crossbar comprises two memristors to implement binary weights. The cells are programmed and read out through CMOS circuitry.

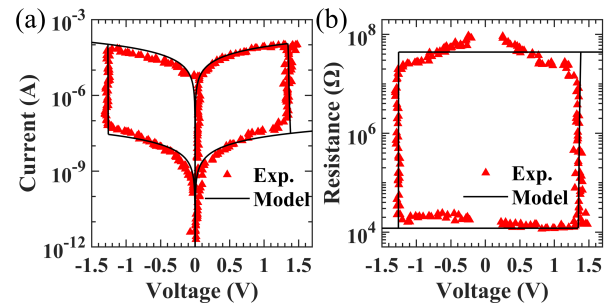


Fig. 2. (a) I-V characteristics and (b) R-V curve of the Al/TiO₂/HfO₂/Pt/Ti/SiO₂/polyimide memristor device used in the array [7]. We use the VTEAM model in Verilog-A to model its behavior [6]. The fitting parameters used are: $V_{\text{off}} = 1.3$ V, $V_{\text{on}} = -1.2$ V, $K_{\text{off}} = 440$ ms⁻¹, $K_{\text{on}} = -3000$ ms⁻¹, $w_{\text{on}} = 0$, $w_{\text{off}} = 10$ nm, $R_{\text{off}} = 44$ MΩ, $R_{\text{on}} = 12$ kΩ, $\alpha_{\text{on}} = 0.5$, and $\alpha_{\text{off}} = 1$.

Boolean logic and neural computation, making it more efficient in terms of data movement.

II. SUB-ARRAY DESIGN FOR LOGIC IMPLEMENTATION

The proposed in-memory circuits (NAND, XOR, and full-adder) were implemented and simulated in Cadence Virtuoso using the VTEAM memristor model and gpdk045 CMOS technology. We use binary memristors (Fig. 2) that exhibit high resistance state (HRS) and low resistance state (LRS) in a crossbar array, with all weights, biases, and inputs represented as binary values $\{-1, 1\}$ [7]. The architecture follows a binary neural network style, with differential 1R

synaptic cells (2 memristors per synapse) encoding single weights for area efficiency.

The activation function employs an op-amp comparator that outputs -1 V when the differential voltage drops below 25 mV. Each memristor connects to ground via a controlling transistor and a resistor R , with currents reflecting the programmed conductance. For reliable operation, R should be close to the LRS value (typically $0.5 \times \text{LRS}$ to LRS), and the HRS/LRS ratio must be at least 10. In this work, the memristor exhibits an HRS of 44 M Ω and an LRS of 12 k Ω , with $R = 10$ k Ω .

1) *NAND Gate*: The gates are implemented using a memristor-based crossbar array, with weights derived from a binary neural network formulation. The weights and bias were derived to satisfy the perceptron condition for the NAND truth table; there are automated ways to implement this as well [4]. Fig. 3 shows the complete NAND schematic with its various parts. For a 2-input NAND gate, the weight vector is $[-1, -1]$ and the bias is 1. The augmented weight vector becomes $[-1, -1, 1]$. This vector is transposed and mapped to a conductance matrix by representing each element as a pair of conductance states (LRS and HRS), where LRS is encoded as 1 and HRS as 0 [2]. A weight of -1 is represented as the pair $(0, 1)$, $+1$ as $(1, 0)$, and 0 as $(0, 0)$, as shown in Fig. 3(b). The conversion from weight and bias matrix to conductance matrix for NAND gate is as follows:

$$\begin{bmatrix} -1 & -1 & 1 \end{bmatrix}^T \Rightarrow \begin{bmatrix} 0 & 1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

This configuration implements the NAND function, where the resulting matrix represents the physical memristor programming states. Each row corresponds to a memristor pair in this NAND implementation. The same principle extends to higher-dimensional implementations where adjacent columns (non-overlapping) form the memristor pairs.

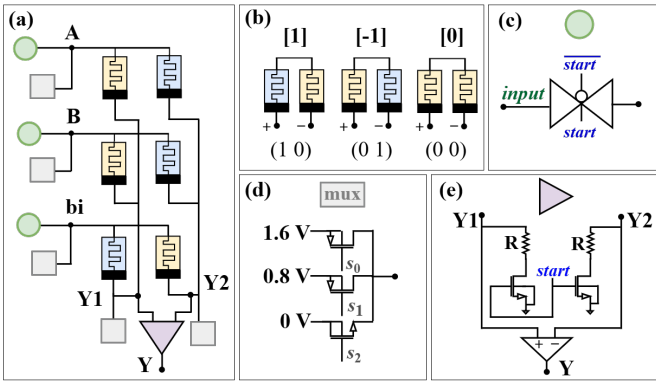


Fig. 3. **NAND gate circuit implementation**: (a) Schematic of sub-array used for NAND. (b) Binary weight encoding in circuit, where blue indicates HRS and yellow indicates LRS state of memristor. (c) Pass-gate circuit used for passing input signal, activated when the processor raises the *start* signal to 1.1 V. (d) Analog 3×1 multiplexer programs voltages of 1.6 V, 0.8 V, and 0 V, through selector signal s_0 , s_1 , and s_2 . (e) Activation-function circuit based on op-amp thresholding.

2) *Programming Procedure*: Programming is implemented via an analog 3×1 multiplexer using a half-bias scheme with the set (0 V, 1.6 V) and reset (1.6 V, 0 V)

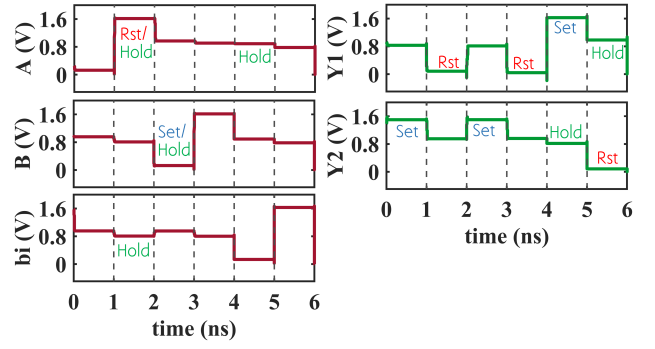


Fig. 4. Programming signals for the NAND gate weights are applied using the half-bias scheme. Rows are pulsed sequentially and all columns of the corresponding row are updated in parallel: columns biased at 1.6 V during a 0 V row pulse are *set* (programmed to LRS), whereas columns biased at 0 V during a 1.6 V row pulse are *reset* (programmed to HRS). A 0.8 V bias on either terminal maintains the memristor state. During 0–2 ns, the memristor at row A, column Y1 is reset to HRS; other devices are programmed similarly.

voltages, where the first value is row voltage and the second is column voltage [8]. A 0.8 V pulse at either row or column maintains the memristor state. The programming scheme operates sequentially by row, where all columns in a single row are programmed concurrently before proceeding to the next row. Columns at 1.6 V during a 0 V row pulse are *set*, while columns at 0 V during a 1.6 V row pulse are *reset*.

The programming signals for NAND are shown in Fig. 4. The small offsets in the voltage levels arise due to changes in path resistance as the devices switch states.

III. RESULTS AND DISCUSSION

1) *NAND Operation*: Memristors were programmed in the initial 12 ns. Computation began at 20 ns by setting the start pin to 1.1 V. All input combinations (00, 01, 11, 10) were applied sequentially, and the output Y matched the theoretical truth table, where 0.8 V and -0.8 V represent logic 1 and logic 0, respectively. $Y1$ and $Y2$ form the differential input to the op-amp.

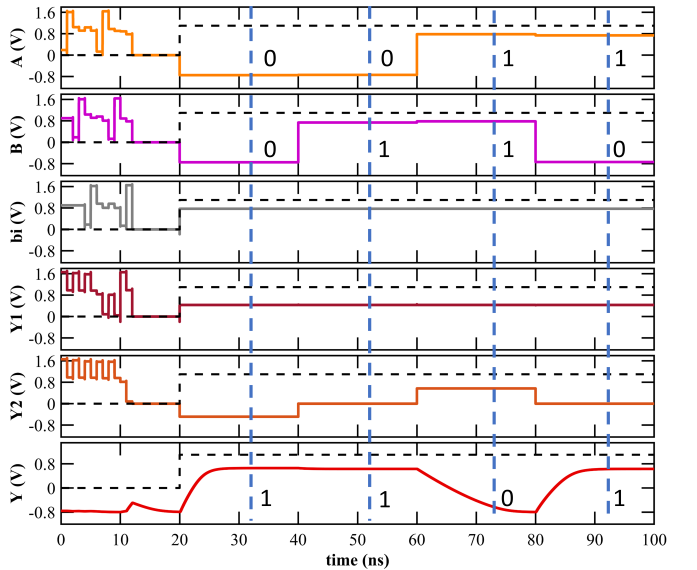


Fig. 5. **NAND operation**: The initial 12 ns are programming pulses. Memristors were first programmed to LRS to simulate the worst case. The dashed line indicates the *start* signal. Input signals are applied to the crossbar nodes A and B through pass gates when the *start* pin is at 1.1 V, with a constant bias $bi = 1$, with logic 1 as 0.8 V and logic 0 as -0.8 V. The output Y follows the NAND truth table.

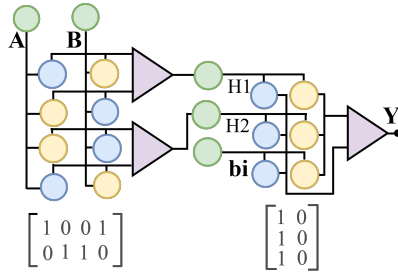


Fig. 6. **XOR circuit:** Schematic of XOR circuit implementation with symbolic memristors, blue indicating HRS and yellow indicating LRS. Input-hidden and hidden-output conductance matrices are shown below the crossbar.

2) **XOR Operation:** Once NAND is implemented, more complex Boolean functions, such as XOR and full-adder, can be realized using the same methodology. The XOR operation ($A \oplus B$) requires a hidden layer with two neurons to capture the necessary nonlinearity, as illustrated in Fig. 6. The corresponding input-hidden and hidden-output conductance matrices are shown in Fig. 6 below the corresponding crossbar. The states of the hidden-layer neurons H1 and H2, along with the final output Y, were verified, as presented in Fig. 7.

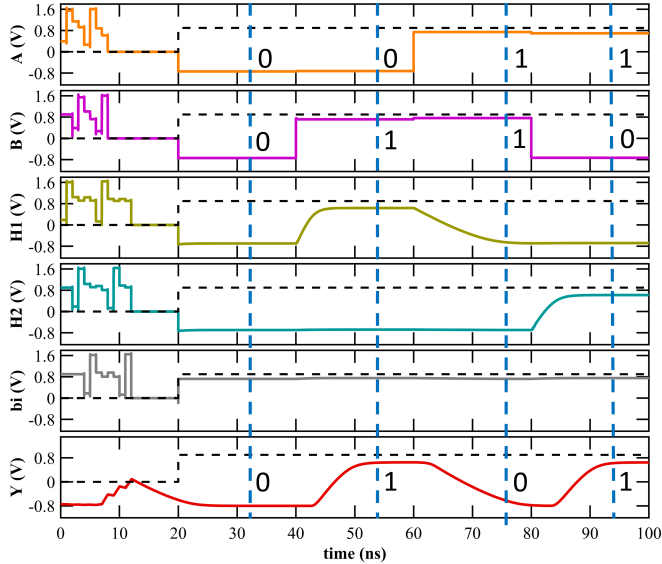


Fig. 7. **XOR operation:** Inputs A and B correspond to crossbar voltages, and a hidden layer with two neurons ($H1$ and $H2$) enables nonlinear XOR computation. The corresponding hidden-layer states and the final XOR output confirm correct functionality.

3) **Full-adder Operation:** The full-adder sum is obtained by cascading two XOR gates: the first computes $A \oplus B$, and the second includes the carry-in C_i to produce $A \oplus B \oplus C_i$ (Fig. 8). The carry-out follows a structure similar to the XOR hidden-output crossbar, with sum and carry appearing at nodes S and C_o , respectively, as shown in Fig. 9.

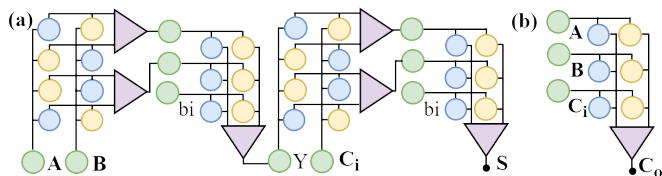


Fig. 8. **Full-adder circuit:** Schematic of the full-adder circuit (a) Sum, S implemented by cascading two XOR circuits and (b) Carry-out, C_o is implemented through a circuit similar to the hidden and output layers of XOR.

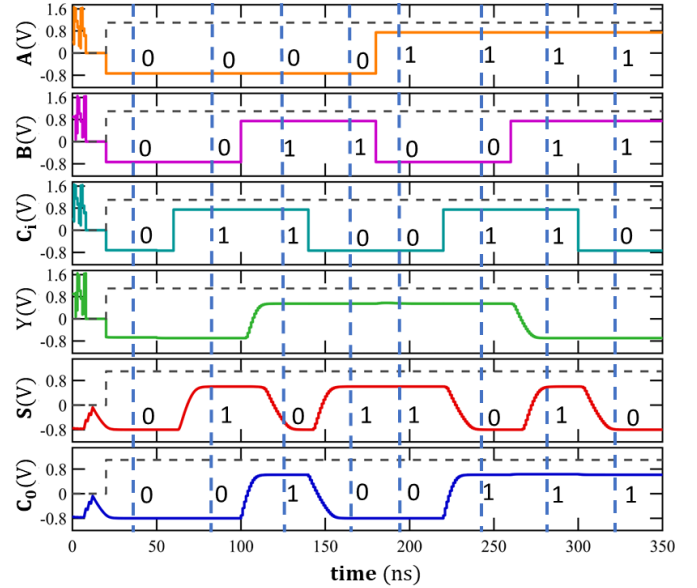


Fig. 9. **Full-adder operation:** The sum (S) is obtained using cascaded XOR gates, with $A \oplus B$ denoted as Y , and the carry-out (C_o) is shown. Inputs A , B , and C_i correspond to crossbar voltages, while S and C_o confirm correct full-adder functionality.

IV. CONCLUSION

In this paper, we demonstrate in-memory Boolean logic computation within a memristor crossbar array structure that has two memristor devices per cell. We performed NAND, XOR, and full-adder computations using a binary neural network-based design approach. Circuit-level SPICE simulations confirmed the correctness of logic computations. The full-adder was constructed by cascading basic XOR and NAND gates, illustrating that arbitrary Boolean functions can be realized from the basic gate designs using this method. This proof-of-concept demonstration could help pave the way for more efficient, non-conventional computing paradigms in future hardware designs.

REFERENCES

- [1] S. Zhang, X. Cui, F. Wei, and X. Cui, "An area-efficient in-memory implementation method of arbitrary boolean function based on sram array," *IEEE Transactions on Computers*, vol. 72, no. 12, pp. 3416–3430, 2023.
- [2] L. Huang, J. Diao, H. Nie, W. Wang, Z. Li, Q. Li, and H. Liu, "Memristor based binary convolutional neural network architecture with configurable neurons," *Frontiers in neuroscience*, vol. 15, p. 639526, 2021.
- [3] N. Lepri, A. Glukhov, L. Cattaneo, M. Farronato, P. Mannocci, and D. Ielmini, "In-memory computing for machine learning and deep learning," *IEEE Journal of the Electron Devices Society*, vol. 11, pp. 587–601, 2023.
- [4] M. Courbariaux, Y. Bengio, and J.-P. David, "Binaryconnect: Training deep neural networks with binary weights during propagations," *Advances in neural information processing systems*, vol. 28, 2015.
- [5] F. Aguirre, A. Sebastian, M. Le Gallo, W. Song, T. Wang, J. J. Yang, W. Lu, M.-F. Chang, D. Ielmini, Y. Yang *et al.*, "Hardware implementation of memristor-based artificial neural networks," *Nature communications*, vol. 15, no. 1, p. 1974, 2024.
- [6] S. Kvaterny, M. Ramadan, E. G. Friedman, and A. Kolodny, "Vteam: A general model for voltage-controlled memristors," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 62, no. 8, pp. 786–790, 2015.
- [7] P. Pal, K.-J. Lee, S. Thunder, S. De, P.-T. Huang, T. Kämpfe, and Y.-H. Wang, "Bending resistant multibit memristor for flexible precision inference engine application," *IEEE Transactions on Electron Devices*, vol. 69, no. 8, pp. 4737–4743, 2022.
- [8] A. Chen, "Analysis of partial bias schemes for the writing of crossbar memory arrays," *IEEE Transactions on Electron Devices*, vol. 62, no. 9, pp. 2845–2849, 2015.